20.5 其他核心通道

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~650k tokens，输出 ~60k tokens（本章合计）

除了 WhatsApp、Telegram、Discord 和 Slack 这四大通道外，OpenClaw 还内置了 Signal、iMessage（含 BlueBubbles）和 WebChat 等通道。这些通道各有独特的技术架构，本节逐一分析。

20.5.1 Signal（signal-cli）

signal-cli 与 JSON-RPC

Signal 是一个以端到端加密著称的通讯应用。由于 Signal 不提供官方 Bot API，OpenClaw 通过 signal-cli 这个第三方命令行工具间接集成。signal-cli 将 Signal 的 Java 客户端库封装为命令行接口，并提供了 HTTP 模式用于程序化调用。

> **衍生解释**：signal-cli 是一个用 Java 编写的 Signal 客户端命令行工具，可以在没有手机的情况下注册 Signal 账户、收发消息。它支持 `daemon` 模式，启动后通过 HTTP 接口暴露 JSON-RPC 2.0 API 和 SSE（Server-Sent Events）事件流。

整体架构如下：

```
signal-cli daemon (Java 进程)
    ├── HTTP JSON-RPC API  ← 发送消息、获取附件
    └── SSE /api/v1/events ← 接收实时事件
         │
         ▼
OpenClaw Signal Monitor
    ├── spawnSignalDaemon()      ← 自动启动 daemon
    ├── signalRpcRequest()       ← JSON-RPC 客户端
    ├── streamSignalEvents()     ← SSE 事件流解析
    └── runSignalSseLoop()       ← 带自动重连的事件循环
```

自动启动 Daemon

OpenClaw 可以自动管理 signal-cli 进程的生命周期：

```
// src/signal/daemon.ts
export function spawnSignalDaemon(opts: SignalDaemonOpts): SignalDaemonHandle {
  const args = buildDaemonArgs(opts);
  // 例如: ["daemon", "--http", "127.0.0.1:8080", "--no-receive-stdout"]

  const child = spawn(opts.cliPath, args, {
    stdio: ["ignore", "pipe", "pipe"],
  });

  // 日志分类：区分普通日志和错误
  child.stderr?.on("data", (data) => {
    for (const line of data.toString().split(/\r?\n/)) {
      const kind = classifySignalCliLogLine(line);
      if (kind === "error") error(`signal-cli: ${line.trim()}`);
      else if (kind === "log") log(`signal-cli: ${line.trim()}`);
    }
  });

  return {
    pid: child.pid,
    stop: () => { if (!child.killed) child.kill("SIGTERM"); }
  };
}
```

signal-cli 的日志分类逻辑会检测 `ERROR` 、 `WARN` 、 `FAILED` 、 `SEVERE` 等关键字来判断日志级别：

```
export function classifySignalCliLogLine(line: string): "log" | "error" | null {
  const trimmed = line.trim();
  if (!trimmed) return null;
  if (/\b(ERROR|WARN|WARNING)\b/.test(trimmed)) return "error";
  if (/\b(FAILED|SEVERE|EXCEPTION)\b/i.test(trimmed)) return "error";
  return "log";
}
```

启动后，OpenClaw 会轮询 `/api/v1/check` 端点等待 daemon 就绪：

```
await waitForSignalDaemonReady({
  baseUrl,
  timeoutMs: startupTimeoutMs,     // 默认 30 秒，最大 120 秒
  logAfterMs: 10_000,              // 10 秒后开始打印等待日志
  pollIntervalMs: 150,             // 每 150ms 轮询一次
  check: async () => {
    const res = await signalCheck(params.baseUrl, 1000);
    return res.ok ? { ok: true } : { ok: false, error: res.error };
  },
});
```

JSON-RPC 客户端

所有与 signal-cli 的交互都通过标准的 JSON-RPC 2.0 协议：

```
// src/signal/client.ts
export async function signalRpcRequest<T>(
  method: string,
  params: Record<string, unknown> | undefined,
  opts: SignalRpcOptions,
): Promise<T> {
  const body = JSON.stringify({
    jsonrpc: "2.0",
    method,              // 例如 "send", "getAttachment", "sendReceipt"
    params,
    id: randomUUID(),    // 每个请求唯一 ID
  });

  const res = await fetchWithTimeout(
    `${baseUrl}/api/v1/rpc`,
    { method: "POST", headers: { "Content-Type": "application/json" }, body },
    opts.timeoutMs ?? 10_000,
  );

  const parsed = JSON.parse(text) as SignalRpcResponse<T>;
  if (parsed.error) {
    throw new Error(`Signal RPC ${parsed.error.code}: ${parsed.error.message}`);
  }
  return parsed.result as T;
}
```

SSE 事件流与自动重连

Signal 的实时消息接收使用 SSE（Server-Sent Events）协议。OpenClaw 实现了完整的 SSE 客户端，包括手动解析事件流：

```
// src/signal/client.ts（简化）
export async function streamSignalEvents(params) {
  const res = await fetch(url, {
    headers: { Accept: "text/event-stream" },
    signal: params.abortSignal,
  });

  const reader = res.body.getReader();
  const decoder = new TextDecoder();
  let buffer = "";
  let currentEvent: SignalSseEvent = {};

  while (true) {
    const { value, done } = await reader.read();
    if (done) break;
    buffer += decoder.decode(value, { stream: true });

    // 按行解析 SSE 协议：event:, data:, id:, 空行分隔事件
    // ...
  }
}
```

> **衍生解释**：SSE（Server-Sent Events）是一种基于 HTTP 的服务器推送协议。服务器通过一个持久 HTTP 连接以文本格式发送事件。每个事件由 `event:` 、 `data:` 、 `id:` 字段组成，以空行分隔。与 WebSocket 不同，SSE 是单向的（服务器→客户端），但更简单且内置自动重连。

外层的 `runSignalSseLoop()` 负责在连接断开时自动重连，使用指数退避策略：

```
// src/signal/sse-reconnect.ts
const DEFAULT_RECONNECT_POLICY: BackoffPolicy = {
  initialMs: 1_000,   // 初始 1 秒
  maxMs: 10_000,      // 最大 10 秒
  factor: 2,          // 翻倍增长
  jitter: 0.2,        // 20% 随机抖动
};

export async function runSignalSseLoop(params) {
  let reconnectAttempts = 0;

  while (!abortSignal?.aborted) {
    try {
      await streamSignalEvents({
        onEvent: (event) => {
          reconnectAttempts = 0;  // 收到事件即重置计数器
          onEvent(event);
        },
      });
      // 正常结束 → 等待后重连
      const delayMs = computeBackoff(reconnectPolicy, ++reconnectAttempts);
      await sleepWithAbort(delayMs, abortSignal);
    } catch (err) {
      // 异常 → 等待更久后重连
      const delayMs = computeBackoff(reconnectPolicy, ++reconnectAttempts);
      await sleepWithAbort(delayMs, abortSignal);
    }
  }
}
```

消息发送

Signal 发送消息同样通过 JSON-RPC，支持电话号码、用户名和群组三种目标类型：

```ts
// src/signal/send.ts
type SignalTarget =
  | { type: "recipient"; recipient: string }   // 电话号码
  | { type: "group"; groupId: string }          // 群组 ID
  | { type: "username"; username: string };      // Signal 用户名

export async function sendMessageSignal(to, text, opts) {
  // Markdown → Signal 富文本样式
  const formatted = markdownToSignalText(message, { tableMode });
  message = formatted.text;
  textStyles = formatted.styles;  // Signal 支持 text-style 属性

  const params = { message, account };
  if (textStyles.length > 0) {
    params["text-style"] = textStyles.map(
      (s) => `${s.start}:${s.length}:${s.style}`
    );
  }

  return signalRpcRequest("send", params, { baseUrl });
}
```

Signal 的文本样式使用偏移量标记（`start:length:style`），例如 `0:5:BOLD` 表示前 5 个字符加粗。

20.5.2 BlueBubbles（iMessage 推荐集成）

BlueBubbles 是 OpenClaw 推荐的 iMessage 集成方式，它通过在 Mac 上运行一个 BlueBubbles 服务器，将 iMessage 的收发能力暴露为 HTTP API。

> **衍生解释**：iMessage 是 Apple 的封闭生态通讯协议，没有公开 API。BlueBubbles 是一个第三方开源项目，它在 macOS 上运行一个服务器进程，监控系统的 Messages 数据库（chat.db），并通过 HTTP/WebSocket 接口提供消息收发能力。这是目前将 iMessage 集成到第三方应用中最可靠的方式之一。

与直接操作 iMessage 数据库的方案不同，BlueBubbles 作为一个独立服务器运行，需要配置 `serverUrl` 和 `password`。

在 OpenClaw 中，BlueBubbles 被实现为**扩展插件**（而非核心通道），位于插件系统中。它的核心配置检查如下：

```
// src/channels/plugins/status-issues/bluebubbles.ts
export function collectBlueBubblesStatusIssues(accounts) {
  for (const entry of accounts) {
    if (!configured) {
      issues.push({
        channel: "bluebubbles",
        kind: "config",
        message: "Not configured (missing serverUrl or password).",
        fix: "Run: openclaw channels add bluebubbles --http-url <server-url> --password
      });
    }
    if (probe && probe.ok === false) {
      issues.push({
        channel: "bluebubbles",
        kind: "runtime",
        message: `BlueBubbles server unreachable${errorDetail}`,
      });
    }
  }
}
```

BlueBubbles 与原生 iMessage（下一小节介绍）的关系是互斥的——当 BlueBubbles 配置存在时，OpenClaw 会优先使用它，并自动跳过原生 iMessage 的启用：

```
# 配置示例
channels:
  bluebubbles:
    serverUrl: "http://mac-mini.local:1234"
    password: "your-bluebubbles-password"
    dmPolicy: "pairing"
```

BlueBubbles 的消息路由系统与其他通道共享相同的 `outbound-session.ts` 逻辑，支持电话号码和 `chat_guid` 两种目标格式。

20.5.3 iMessage Legacy（macOS 原生 imsg）

当无法使用 BlueBubbles 时，OpenClaw 提供了直接通过 macOS 原生接口操作 iMessage 的方案。这种方式需要在 macOS 机器上运行一个名为 `imsg` 的 RPC 命令行工具。

imsg RPC 客户端

`imsg` 是一个以 JSON-RPC 2.0 协议通信的命令行进程，OpenClaw 通过 stdin/stdout 管道与之交互：

```
// src/imessage/client.ts
export class IMessageRpcClient {
  private child: ChildProcessWithoutNullStreams | null = null;
  private readonly pending = new Map<string, PendingRequest>();
  private nextId = 1;

  async start(): Promise<void> {
    const args = ["rpc"];
    if (this.dbPath) args.push("--db", this.dbPath);

    const child = spawn(this.cliPath, args, {
      stdio: ["pipe", "pipe", "pipe"],  // stdin, stdout, stderr 都接管
    });
    this.child = child;
    this.reader = createInterface({ input: child.stdout });

    // 逐行读取 JSON-RPC 响应
    this.reader.on("line", (line) => {
      this.handleLine(line.trim());
    });
  }

  async request<T>(method: string, params?, opts?): Promise<T> {
    const payload = { jsonrpc: "2.0", id: this.nextId++, method, params };
    const line = `${JSON.stringify(payload)}\n`;

    return new Promise((resolve, reject) => {
      const timer = setTimeout(() => reject(new Error("timeout")), timeoutMs);
      this.pending.set(String(id), { resolve, reject, timer });
    });

    this.child.stdin.write(line);  // 写入 stdin
  }
}
```

这种基于 stdio 管道的 JSON-RPC 通信模式在系统编程中很常见，它的优点是：

- 不需要网络端口，避免端口冲突

- 进程生命周期由父进程直接管理
- 支持双向异步通信（请求/响应 + 通知）

消息监听与订阅

iMessage 的消息监听通过 `watch.subscribe` RPC 方法实现：

```
// src/imessage/monitor/monitor-provider.ts
const client = await createIMessageRpcClient({
  cliPath,
  dbPath,
  runtime,
  onNotification: (msg) => {
    if (msg.method === "message") {
      void handleMessage(msg.params);  // 收到新消息
    } else if (msg.method === "error") {
      runtime.error?.(`imessage: watch error ${JSON.stringify(msg.params)}`);
    }
  },
});

// 订阅新消息通知
const result = await client.request("watch.subscribe", {
  attachments: includeAttachments,
});
subscriptionId = result?.subscription ?? null;

// 等待连接关闭（阻塞直到进程退出或 abort）
await client.waitForClose();
```

`imsg` 工具在后台持续监控 macOS 的 Messages 数据库（`~/Library/Messages/chat.db`），当检测到新消息时通过 JSON-RPC 通知推送给 OpenClaw。

远程 Mac 支持

OpenClaw 支持通过 SSH 远程访问另一台 Mac 上的 iMessage。它会自动从 CLI 路径检测 SSH 包装脚本：

```
async function detectRemoteHostFromCliPath(cliPath: string) {
  const content = await fs.readFile(expanded, "utf8");

  // 匹配: exec ssh -T openclaw@192.168.64.3 /opt/homebrew/bin/imsg "$@"
  const userHostMatch = content.match(/\bssh\b[^\n]*?\s+([a-zA-Z0-9._-]+@[a-zA-Z0-9._-]
  if (userHostMatch) return userHostMatch[1];

  // 匹配: exec ssh -T mac-mini imsg "$@"
  const hostOnlyMatch = content.match(/\bssh\b[^\n]*?\s+([a-zA-Z][a-zA-Z0-9._-]*)\s+\S*
  return hostOnlyMatch?.[1];
}
```

检测到远程主机后，它会被记录到消息上下文的 `MediaRemoteHost` 字段中，供媒体处理系统使用（远程文件路径需要通过 SSH 拉取）。

回声检测

iMessage 通道有一个独特的问题：通过 `imsg` 发送的消息也会被监控到（因为 Messages 数据库会写入所有消息）。OpenClaw 使用 `SentMessageCache` 来过滤这些"回声"：

```
class SentMessageCache {
  private cache = new Map<string, number>();
  private readonly ttlMs = 5000;  // 5 秒窗口

  remember(scope: string, text: string): void {
    this.cache.set(`${scope}:${text.trim()}`, Date.now());
  }

  has(scope: string, text: string): boolean {
    const timestamp = this.cache.get(key);
    return timestamp ? (Date.now() - timestamp <= this.ttlMs) : false;
  }
}
```

每次发送消息时记录到缓存，收到新消息时检查是否在 5 秒窗口内匹配。Scope 按对话隔离，避免不同对话中相同文本被误判。

消息载荷

iMessage 的消息格式包含丰富的元数据：

```typescript
// src/imessage/monitor/types.ts
export type IMessagePayload = {
  id?: number;
  chat_id?: number;              // macOS 内部对话 ID
  sender?: string;               // 电话号码或 Apple ID
  is_from_me?: boolean;          // 是否自己发的
  text?: string;
  reply_to_id?: number | string; // 引用回复
  reply_to_text?: string;
  reply_to_sender?: string;
  created_at?: string;
  attachments?: IMessageAttachment[];
  chat_identifier?: string;      // 如 "iMessage;-;+1234567890"
  chat_guid?: string;            // 全局唯一 ID
  chat_name?: string;            // 群组名称
  participants?: string[];       // 群组成员列表
  is_group?: boolean;
};
```

OpenClaw 对 iMessage 目标的寻址支持多种格式：

格式	示例	说明
电话号码	`+15551234567`	最常见的方式
chat_id	`imessage:chat:42`	macOS 内部数据库 ID
chat_guid	`imessage:chat_guid:iMessage;-;+155...`	全局唯一
chat_identifier	`imessage:chat_identifier:+155...`	对话标识符

20.5.4 WebChat（Gateway 内置 Web 聊天）

WebChat 是 OpenClaw Gateway 内置的 Web 聊天界面，它不依赖任何第三方服务，而是通过 Gateway 的 WebSocket 协议直接与 Agent 通信。

架构特点

与其他通道通过外部 API/SDK 接收消息不同，WebChat 是 Gateway 协议的原生组成部分：

```
浏览器 WebChat UI
    │
    ▼
WebSocket 连接
    │
    ▼
Gateway WebSocket 服务器
    ├── ws-connection.ts          ← 连接管理
    ├── message-handler.ts        ← 消息路由
    └── server-methods/chat.ts    ← chat.send 方法
         │
         ▼
dispatchInboundMessage()         ← 统一入站分发
```

WebChat 的消息通道标识为 `"webchat"`，定义在：

```
// src/utils/message-channel.ts
export const INTERNAL_MESSAGE_CHANNEL = "webchat" as const;
```

WebSocket 连接

WebChat 客户端通过 WebSocket 连接到 Gateway 服务器。连接建立后，通过 `connect` 消息完成握手：

```
// src/gateway/server/ws-connection/message-handler.ts
`webchat connected conn=${connId} remote=${remoteAddr} client=${clientLabel} ` +
`${connectParams.client.mode} v${connectParams.client.version}`
```

客户端信息中包含模式和版本号：

```
// src/gateway/protocol/client-info.ts
export const GATEWAY_CLIENT_MODE = {
  WEBCHAT_UI: "webchat-ui",
  WEBCHAT: "webchat",
  // ...
};
```

chat.send 方法

WebChat 的消息发送使用 Gateway 的 RPC 协议中的 `chat.send` 方法。它直接调用 `dispatchInboundMessage()` 分发到 Agent：

```
// src/gateway/server-methods/chat.ts（概念简化）
const ctxPayload = {
  Body: messageText,
  Provider: "webchat",
  Surface: "webchat",
  ChatType: "direct",
  // ...
};

const { queuedFinal } = await dispatchInboundMessage({
  ctx: ctxPayload,
  cfg,
  dispatcher,
  replyOptions: { /* ... */ },
});
```

与其他通道的差异

WebChat 与其他通道有几个重要差异：

特性	WebChat	其他通道
消息来源	直接 WebSocket RPC	外部 API/SDK
持久化	Gateway 管理的会话	外部平台的消息历史
实时性	原生实时	依赖平台推送延迟
出站投递	不支持 `openclaw agent` 主动投递	支持
Heartbeat 配置	使用 `channels.defaults`	有独立的 per-channel 配置

WebChat 不支持通过 CLI 的 `openclaw agent` 命令主动投递消息：

```
// src/infra/outbound/targets.ts
`Delivering to WebChat is not supported via ${formatCliCommand("openclaw agent")}; ` +
`use WhatsApp/Telegram or run with --deliver=false.`
```

Heartbeat 可见性

WebChat 有独特的 Heartbeat 可见性控制——由于 WebChat 通常用于开发调试，默认会抑制 Heartbeat 的广播：

```
// src/infra/heartbeat-visibility.ts
if (channel === "webchat") {
  // webchat 使用 channels.defaults.heartbeat 配置
  // 没有 per-channel 配置
}
```

会话与队列

WebChat 的会话与队列系统也有特殊处理：

```
// src/config/sessions/group.ts
const getGroupSurfaces = () => new Set([...listDeliverableMessageChannels(), "webchat"]

// src/config/types.queue.ts
type QueueConfig = {
  webchat?: QueueMode;  // 独立的 webchat 队列配置
};
```

WebChat 被视为可投递通道组（Group Surfaces）的一员，这意味着 Agent 在处理来自 WebChat 的消息时，可以访问与其他通道相同的上下文信息和工具集。

本节小结

1. **Signal 通道通过 signal-cli 间接集成**，使用 JSON-RPC 2.0 协议发送消息，SSE 协议接收事件。OpenClaw 可以自动启动和管理 signal-cli daemon 进程，并内置了带指数退避的 SSE 重连循环。
2. **BlueBubbles 是 OpenClaw 推荐的 iMessage 集成方案**，通过在 Mac 上运行 BlueBubbles 服务器暴露 HTTP API。它作为扩展插件实现，与原生 iMessage 互斥。
3. **iMessage Legacy 通过 `imsg` RPC 工具**直接操作 macOS Messages 数据库，使用 stdio 管道的 JSON-RPC 通信。独特的设计包括远程 Mac 的 SSH 支持和 5 秒窗口的回声检测。
4. **WebChat 是 Gateway 内置的 Web 聊天**，不依赖外部服务，直接通过 WebSocket RPC 与 Agent 通信。它是唯一不支持主动出站投递的通道，但拥有最低的延迟和最简单的配置。

5. **这些通道共同体现了 OpenClaw 的通道抽象设计**——无论底层协议是 HTTP API、SSE、stdio 管道还是 WebSocket，所有消息最终都被规范化为统一的 `MsgContext` 结构，交由 `dispatchInboundMessage()` 处理。